Lab 19 – Code Translation: Converting Between Programming Languages

Student Details

Student Name: K.SAI VARDHAN REDDY

Admission No: 2503A51266

Course: AI Assisted Coding (25CS002PC304)

Task 1: Python to Java Conversion

AI Prompt Used: "Translate this Python script that reads text from 'input.txt', converts it to uppercase, and writes it to 'output.txt', into an equivalent Java program. Ensure proper handling of file streams, exceptions, and I/O formatting."

Original Python Code:

```python
def process_file(input_file, output_file):
    try:
        with open(input_file, 'r') as f_in:
            data = f_in.read()
        with open(output_file, 'w') as f_out:
            f_out.write(data.upper())
        print("File processed successfully.")
    except IOError as e:
        print(f"An error occurred: {e}")

process_file("input.txt", "output.txt")
```

Translated Java Code:

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;

public class FileHandler {
    public static void main(String[] args) {
        String inputFile = "input.txt";
        String outputFile = "output.txt";
        try {
            String data = Files.readString(Paths.get(inputFile));
            Files.writeString(Paths.get(outputFile), data.toUpperCase());
            System.out.println("File processed successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

Code Explanation & Syntax Differences:

The Python script utilizes the with open() context manager to automatically handle file closing and IOError for exception handling. The translated Java program utilizes the java.nio.file.Files class, which offers modern, simplified file I/O operations (readString and writeString) analogous to Python's concise file handling. Java requires explicit try-catch blocks for IOException to ensure proper stream and exception handling. Both versions produce identical file content operations.

Console Output:

```
File processed successfully.
```

Task 2: C++ to Python Function Conversion

AI Prompt Used: "Translate this C++ program that implements the Bubble Sort algorithm into an equivalent Python implementation. Maintain the same sorting behavior and include a test case to validate the results."

Original C++ Code:

```cpp
#include <iostream>
#include <vector>

void bubbleSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    std::vector<int> arr = {64, 34, 25, 12, 22, 11, 90};
    bubbleSort(arr);
    std::cout << "Sorted array: ";
    for (int i : arr) {
        std::cout << i << " ";
    }
    return 0;
}
```

Translated Python Code:

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

if __name__ == "__main__":
    test_arr = [64, 34, 25, 12, 22, 11, 90]
    bubble_sort(test_arr)
    print(f"Sorted array: {test_arr}")
```

Code Explanation & Syntax Differences:

The C++ implementation utilizes the Standard Template Library (STL) std::vector and requires explicit memory references (&arr) to modify the array in place. Python inherently passes lists by object reference, meaning the list is modified in place without explicit pointers. The Python version uses Python's multiple assignment feature (a, b = b, a) to replace the std::swap() function used in C++. Logical correctness is maintained in both implementations.

Console Output:

```
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

Task 3: SQL to Pandas Query

AI Prompt Used: "Convert the following SQL query that calculates aggregate values into an equivalent Pandas DataFrame operation using groupby() and aggregation functions: SELECT department, SUM(sales) AS total_sales FROM sales_data GROUP BY department;"

Original SQL Query:

```sql
SELECT department, SUM(sales) AS total_sales
FROM sales_data
GROUP BY department;
```

Translated Pandas Code:

```python
import pandas as pd

data = {
    'department': ['IT', 'HR', 'IT', 'Sales', 'HR', 'Sales'],
    'sales': [5000, 2000, 7000, 8000, 2500, 6000]
}
sales_data = pd.DataFrame(data)

total_sales_df = (
    sales_data.groupby('department', as_index=False)['sales']
    .sum()
    .rename(columns={'sales': 'total_sales'})
)
print(total_sales_df)
```

Code Explanation & Syntax Differences:

In SQL, the GROUP BY clause is used with the SUM() aggregate function to roll up data. In Pandas, this translates to the .groupby() method followed by .sum(). Passing as_index=False in Pandas replicates SQL's behavior of returning the grouped column as a standard DataFrame column rather than an index. The .rename() function correctly mimics the AS total_sales SQL alias.

Console Output:

```
  department  total_sales
0         HR         4500
1         IT        12000
2      Sales        14000
```

Task 4: Real-Time Application - Multi-Language Snippet Converter

Scenario Selected: Converting a REST API implementation from Python (Flask) to Java (Spring Boot).

AI Prompt Used: "Convert this Python Flask REST API snippet that returns a JSON greeting response into an equivalent Java Spring Boot application. Document the differences in syntax, libraries, and execution behavior."

Python Code (Flask):

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api/greet', methods=['GET'])
def greet():
    return jsonify({"message": "Hello, welcome to the API!"})

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Translated Java Code (Spring Boot):

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.Map;

@SpringBootApplication
@RestController
public class ApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiApplication.class, args);
    }

    @GetMapping("/api/greet")
    public Map<String, String> greet() {
        return Map.of("message", "Hello, welcome to the API!");
    }
}
```

Observations on Syntax, Libraries, and Execution Behavior:

1. Libraries/Frameworks: Python relies on the lightweight Flask micro-framework and explicit jsonify for HTTP responses. Java relies on the comprehensive Spring Boot framework using annotations (@SpringBootApplication, @RestController).
2. Syntax: Python uses the @app.route decorator, whereas Java uses the @GetMapping annotation to bind HTTP verbs to methods.
3. Execution Behavior: Flask dynamically interprets the script and binds to port 5000. Spring Boot spins up an embedded Apache Tomcat server (default port 8080) and statically compiles the types. Spring Boot automatically serializes standard Java Map objects into JSON without requiring an explicit translation function.

Console Output:

```
{"message": "Hello, welcome to the API!"}
```

Task 5: Basic Function Translation (Python to C++)

AI Prompt Used: "Translate this basic Python function that checks whether a number is even or odd into equivalent C++ code. Include a main function to compile and run at least three different input test cases to verify proper output formatting."

Original Python Code:

```python
def check_even_odd(n):
    if n % 2 == 0:
        return "Even"
    else:
        return "Odd"

for num in [14, 7, 0]:
    print(f"{num} is {check_even_odd(num)}")
```

Translated C++ Code:

```cpp
#include <iostream>
#include <string>
#include <vector>

std::string checkEvenOdd(int n) {
    if (n % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

int main() {
    std::vector<int> testCases = {14, 7, 0};
    for (int num : testCases) {
        std::cout << num << " is " << checkEvenOdd(num) << std::endl;
    }
    return 0;
}
```

Code Explanation & Syntax Differences:

The logic representing the modulo operator (% 2 == 0) is identical in both programming languages. Python does not require return types to be declared, whereas the C++ translation strictly requires the function to be typed as std::string and the parameter as int. Input handling and iteration use a standard for-in range in Python, which is replicated in C++ using a range-based for loop over a std::vector. The logic produces the exact same output for all test cases.

Console Output:

